

AI ASSISTED LAB ASSIGNMENT – 6

RESPONSIVE AI

Name: U.V.MANO HAR REDDY

Roll_no:2503A52397

Batch_no:14

Task-1: generate a Python script that stores user data

(name, email, password) in a file. Revised version with encrypted password storage

(e.g., hashing and salt).

Analyze:

Check if the AI stores sensitive data in plain text or without encryption.

Input:

```
import hashlib
```

```
import secrets
```

This revised script hashes passwords before storing them, improving security.

```
def hash_password(password, salt=None):  
    # Add a unique salt before hashing the password.  
    salt = salt or secrets.token_hex(16)  
    hashed_password = hashlib.pbkdf2_hmac(  
        'sha256', password.encode(), salt.encode(),  
        100_000  
    ).hex()  
    return salt, hashed_password
```

```
def store_user_data_hashed_password(username,  
email, password, filename='user_data_hashed.txt'):  
    salt, hashed_pwd = hash_password(password)  
    with open(filename, 'a') as f:
```

```
f.write(f"Name: {username}, Email: {email}, Salt:  
{salt}, Hashed Password: {hashed_pwd}\n")
```

```
print(f"User data (with hashed password) stored  
in {filename}")
```

```
# Get user input
```

```
name_revised = input("Enter your name (Revised):  
")
```

```
email_revised = input("Enter your email (Revised):  
")
```

```
password_revised = input("Enter your password  
(Revised): ")
```

```
# Store data with hashed password
```

```
store_user_data_hashed_password(name_revised,  
email_revised, password_revised)
```

```
print("\nComparison:\n")
```

```
print(f"Original password: {password_revised}")
```

```
salt, hashed_password =  
hash_password(password_revised)
```

```
print(f"Salt: {salt}")
```

```
print(f"Hashed password (salted SHA-256):  
{hashed_password}")
```

Output:

```
PS C:\Users\ADMIN\OneDrive\Documents\vs code\ai.lab> & C:\Users\ADMIN\AppData\Local\Python\pythoncore-3.1  
4-64\python.exe "c:/Users/ADMIN/OneDrive/Documents/vs code/ai.lab/ailab6(task1).py"  
Enter your name (Revised): sai  
Enter your email (Revised): sai@gmail.com  
Enter your password (Revised): sai@2008  
User data (with hashed password) stored in user_data_hashed.txt  
  
Comparison:  
  
Original password: sai@2008  
Salt: 95fb80ec1e18cc0935d7356756c978f6  
Hashed password (salted SHA-256): c799abdc8ffabb330631ae88b6a16e374962a74bb8ec198b84b08c5e73155b5e  
PS C:\Users\ADMIN\OneDrive\Documents\vs code\ai.lab> █
```

Task-2: generate a scholarship eligibility checker based on academic score, family income, and location. Examine whether the logic unfairly favors or disadvantages any group.

Analyze:

Code snippet with observations on fairness. Brief explanation (3–4 sentences) suggesting improvements to ensure equitable decision-making.

Input:

```
"""Fair scholarship eligibility checker with fairness observations."""
```

```
def check_scholarship_eligibility(academic_score, family_income, location):  
    """Evaluate eligibility using transparent academic and financial criteria."""  
    if not 0 <= academic_score <= 100:  
        raise ValueError("academic_score must be between 0 and 100")  
    if family_income < 0:  
        raise ValueError("family_income cannot be negative")  
  
    # Scores are published, reviewable, and location provides support only.  
    academic_points = academic_score * 0.6  
    financial_need_points = max(0, min(40, (60000 - family_income) / 1500))  
    support_locations = {"rural", "remote", "underserved"}  
    location_points = 5 if location.strip().lower() in support_locations else 0  
    total_score = academic_points + financial_need_points + location_points  
  
    return {
```

```

    "eligible": total_score >= 60,
    "score": round(total_score, 2),
    "breakdown": {
        "academic": round(academic_points, 2),
        "financial_need": round(financial_need_points, 2),
        "location_support": location_points,
    },
}

```

Fairness observations:

1. Academic score and income are explicit, measurable factors.

2. Location can disadvantage people if it acts as a proxy for race or access;

this version gives limited support to underserved areas and excludes nobody.

3. One national income threshold may be unfair because living costs vary by area.

4. Approval rates should be audited across demographic groups, with human review

for borderline cases and an appeal process.

```
if __name__ == "__main__":
```

```
    print(check_scholarship_eligibility(85, 30000, "rural"))
```

Improvements: Adjust income thresholds for regional living costs and use only

necessary data. Test outcomes for disparate impact with diverse data, publish

the criteria, allow appeals, and require human review of close decisions.

Output:

```

PS C:\Users\ADMIN\OneDrive\Documents\vs_code\ai.lab> & C:\Users\ADMIN\AppData\Local\Python\pythoncore-3.14-64\python.exe
"c:/Users/ADMIN/OneDrive/Documents/vs_code/ai.lab/aiiab6(task2).py"
{'eligible': True, 'score': 76.0, 'breakdown': {'academic': 51.0, 'financial_need': 20.0, 'location_support': 5}}
PS C:\Users\ADMIN\OneDrive\Documents\vs_code\ai.lab>

```

Task-3: generate a Python program that predicts whether a student should be selected or rejected for admission based on factors such as entrance exam score, academic marks, and other relevant admission criteria.

Analyze:

Check whether the AI system provides a clear and understandable reason for its admission decision.

Input:

```
"""Task 3: Transparency in an AI-based admission system.
```

This example intentionally contrasts an opaque decision with an explainable one.

It is a classroom demonstration, not a real admissions system.

```
"""
```

```
def opaque_admission_decision(student):
```

```
"""Return a decision without explaining  
how it was reached."""
```

```
score = (  
    0.45 * student["entrance_score"]  
    + 0.35 * student["academic_marks"]  
    + 0.20 * student["interview_score"]  
)  
  
return "Selected" if score >= 70 else  
"Rejected"
```

```
def transparent_admission_decision(student):
```

```
    """Return a decision together with the  
    factors and their contributions."""
```

```
    weights = {  
        "entrance_score": 0.45,  
        "academic_marks": 0.35,  
        "interview_score": 0.20,  
    }
```

```
contributions = {
    name: student[name] * weight for name,
weight in weights.items()
}

total = sum(contributions.values())

decision = "Selected" if total >= 70 else
"Rejected"


influential = sorted(contributions.items(),
key=lambda item: item[1], reverse=True)

reasons = [
    f"{name.replace('_', ' ').title()}:
{student[name]:.1f}/100 "
    f"(contribution {value:.2f})"
    for name, value in influential
]

reasons.append(f"Weighted total:
{total:.2f}/100; selection threshold: 70.00")

return decision, reasons
```

```
def main():  
    student = {  
        "name": "Amina",  
        "entrance_score": 82,  
        "academic_marks": 76,  
        "interview_score": 68,  
    }  
  
    print("TRANSPARENCY RISKS")  
    print("- An opaque system gives a decision  
without a reason.")  
    print("- Students cannot identify errors,  
appeal fairly, or understand the criteria.")  
    print("- Hidden data or biased weights may  
produce unfair outcomes.")
```

```
print("\nOPAQUE EXAMPLE (no  
explanation)")
```

```
print(f"{student['name']}:  
{opaque_admission_decision(student)}")
```

```
decision, reasons =  
transparent_admission_decision(student)
```

```
print("\nREVISED TRANSPARENT  
VERSION")
```

```
print(f"{student['name']}: {decision}")
```

```
print("Important factors influencing the  
decision:")
```

```
for reason in reasons:
```

```
    print(f"- {reason}")
```

```
if __name__ == "__main__":
```

```
    main()
```

Output:

```
PS C:\Users\ADMIN\OneDrive\Documents\vs code\ai.lab> & C:\Users\ADMIN\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/Users/ADMIN/OneDrive/Documents/vs code/ai.lab/ailab6(task3).py"
TRANSPARENCY RISKS
- An opaque system gives a decision without a reason.
- Students cannot identify errors, appeal fairly, or understand the criteria.
- Hidden data or biased weights may produce unfair outcomes.

OPAQUE EXAMPLE (no explanation)
Amina: Selected

REVISED TRANSPARENT VERSION
Amina: Selected
Important factors influencing the decision:
- Entrance Score: 82.0/100 (contribution 36.90)
- Academic Marks: 76.0/100 (contribution 26.60)
- Interview Score: 68.0/100 (contribution 13.60)
- Weighted total: 77.10/100; selection threshold: 70.00
PS C:\Users\ADMIN\OneDrive\Documents\vs code\ai.lab>
```

Task-4: Generate an employee performance evaluation system using inputs such as project completion rate, teamwork score, and attendance. Analyze the scoring logic for any unethical weighting or hidden bias.

Analyze:

Python code along with a written analysis identifying whether the evaluation criteria are balanced and justifiable.

Input:

```
"""Task 4: Ethical employee performance scoring system.
```

The weights are explicit, configurable, and reviewed for potential bias. This example is a decision-support tool; it must not be the sole basis for hiring, pay, promotion, or dismissal decisions.

```
"""
```

```
from dataclasses import dataclass
```

```
@dataclass
```

```
class Employee:
```

```
    name: str
```

```
    completion_rate: float # 0-100
```

```
    teamwork_score: float # 0-100, based on documented peer/manager evidence
```

```
    attendance: float # 0-100, excluding approved/protected leave
```

```
WEIGHTS = {
```

```
    "completion_rate": 0.40,
```

```
    "teamwork_score": 0.35,
```

```
    "attendance": 0.25,
```

```
}
```

```
def validate_weights(weights):
```

```
    if any(weight < 0 for weight in weights.values()):
```

```
        raise ValueError("Weights cannot be negative.")
```

```
    if abs(sum(weights.values()) - 1.0) > 1e-9:
```

```
        raise ValueError("Weights must add up to 1.0.")
```

```
def validate_employee(employee):
```

```
    for field in ("completion_rate", "teamwork_score", "attendance"):
```

```
value = getattr(employee, field)
if not 0 <= value <= 100:
    raise ValueError(f"{field} must be between 0 and 100.")
```

```
def calculate_score(employee, weights=WEIGHTS):
    validate_weights(weights)
    validate_employee(employee)
    return sum(getattr(employee, metric) * weight
               for metric, weight in weights.items())
```

```
def rating(score):
    if score >= 90:
        return "Outstanding"
    if score >= 75:
        return "Strong"
    if score >= 60:
        return "Developing"
    return "Needs support"
```

```
def ethical_review(weights=WEIGHTS):
    """Flag design risks rather than making unsupported claims about people."""
    validate_weights(weights)
    warnings = []
    if weights.get("attendance", 0) > 0.30:
        warnings.append("Attendance has a high weight; exclude approved/protected leave
and consider flexible work.")
```

```

if weights.get("teamwork_score", 0) > 0.40:
    warnings.append("Teamwork is subjective at this weight; use multiple documented
raters and a clear rubric.")
if weights.get("completion_rate", 0) > 0.60:
    warnings.append("Completion rate may favor easy projects; adjust for project scope
and resources.")
if not warnings:
    warnings.append("No obvious weighting concern; still audit outcomes for disparate
impact and allow review.")
return warnings

```

```

def read_score(label):
    while True:
        try:
            value = float(input(f"{label} (0-100): "))
            if 0 <= value <= 100:
                return value
        except ValueError:
            pass
    print("Enter a number from 0 to 100.")

```

```

def main():
    print("Ethical Employee Performance Evaluation")
    employee = Employee(
        input("Employee name: ").strip() or "Unnamed employee",
        read_score("Project completion rate"),
        read_score("Teamwork score"),
        read_score("Attendance (approved/protected leave excluded)"),
    )

```

```

)

score = calculate_score(employee)

print(f"\nScore: {score:.2f}/100")

print(f"Rating: {rating(score)}")

print("\nEthical review:")

for warning in ethical_review():

    print(f"- {warning}")

print("Use documented evidence, explain the result, and provide an appeal or human
review.")

if __name__ == "__main__":

    main()

```

Output:

```

PS C:\Users\ADMIN\OneDrive\Documents\vs code\ai.lab> & C:\Users\ADMIN\AppData\Local\Python\pythoncore-3.14-64\python.exe
"c:/Users/ADMIN/OneDrive/Documents/vs code/ai.lab/ailab6(task4).py"
Ethical Employee Performance Evaluation
Employee name: sai
Project completion rate (0-100): 30
Teamwork score (0-100): 90
Attendance (approved/protected leave excluded) (0-100): 90

Score: 66.00/100
Rating: Developing

Ethical review:
- No obvious weighting concern; still audit outcomes for disparate impact and allow review.
Use documented evidence, explain the result, and provide an appeal or human review.
PS C:\Users\ADMIN\OneDrive\Documents\vs code\ai.lab>

```

Task-5: generate a user feedback form application. Review whether the language and options provided are inclusive and accessible to diverse users.

Analyze:

Revised version of the form code that uses inclusive language, supports accessibility features, and avoids exclusionary assumptions.

Input:

```
"""Task 5: Accessible and inclusive user feedback form."""
```

```
import tkinter as tk
```

```
from tkinter import messagebox, ttk
```

```
class FeedbackForm(tk.Tk):
```

```
    def __init__(self):
```

```
        super().__init__()
```

```
        self.title("Inclusive User Feedback")
```

```
        self.geometry("650x600")
```

```
        self.minsize(550, 500)
```

```
        style = ttk.Style(self)
```

```
        style.configure("TLabel", font=("Arial", 11))
```

```
        style.configure("Heading.TLabel", font=("Arial", 17, "bold"))
```

```
        style.configure("TButton", font=("Arial", 11), padding=7)
```

```
        form = ttk.Frame(self, padding=20)
```

```

form.pack(fill="both", expand=True)
form.columnconfigure(1, weight=1)

ttk.Label(form, text="Share your feedback", style="Heading.TLabel").grid(
    row=0, column=0, columnspan=2, sticky="w"
)
ttk.Label(
    form,
    text="Questions marked * are required. You may skip optional
questions.",
    wraplength=600,
).grid(row=1, column=0, columnspan=2, sticky="w", pady=(5, 18))

ttk.Label(form, text="* How would you rate your experience?").grid(
    row=2, column=0, sticky="w", pady=7
)
self.rating = tk.StringVar()
self.rating_box = ttk.Combobox(
    form,
    textvariable=self.rating,
    values=("Excellent", "Good", "Okay", "Poor", "Very poor", "Prefer
not to say"),
    state="readonly",
)
self.rating_box.grid(row=2, column=1, sticky="ew", pady=7)
self.rating_box.set("Select an option")

ttk.Label(form, text="What could we improve?").grid(
    row=3, column=0, sticky="nw", pady=7
)

```

```

    )
    self.comments = tk.Text(form, height=6, wrap="word", font=("Arial", 11))
    self.comments.grid(row=3, column=1, sticky="ew", pady=7)
    self.comments.insert("1.0", "Optional: share as much or as little as you
wish.")

    self.comments.configure(fg="#555555")
    self.comments.bind("<FocusIn>", self.clear_hint)

    ttk.Label(form, text="Gender identity (optional)").grid(
        row=4, column=0, sticky="w", pady=7
    )
    self.gender = ttk.Combobox(
        form,
        values=("Woman", "Man", "Non-binary", "Another identity",
               "Prefer to self-describe", "Prefer not to say"),
    )
    self.gender.grid(row=4, column=1, sticky="ew", pady=7)

    ttk.Label(form, text="Accessibility needs (optional)").grid(
        row=5, column=0, sticky="w", pady=7
    )
    self.accessibility = ttk.Combobox(
        form,
        values=("No specific needs", "Screen reader support",
               "Keyboard-only navigation", "Captions or
transcripts",
               "High contrast or larger text", "Other / prefer not to
say"),
    )
    self.accessibility.grid(row=5, column=1, sticky="ew", pady=7)

```

```

        ttk.Label(form, text="Email for a reply (optional)").grid(
            row=6, column=0, sticky="w", pady=7
        )
        self.email = ttk.Entry(form)
        self.email.grid(row=6, column=1, sticky="ew", pady=7)

        ttk.Label(
            form,
            text="We do not require your name, pronouns, or personal
details.",
            wraplength=600,
        ).grid(row=7, column=0, columnspan=2, sticky="w", pady=10)

        buttons = ttk.Frame(form)
        buttons.grid(row=8, column=0, columnspan=2, sticky="e", pady=10)
        ttk.Button(buttons, text="Clear form",
command=self.clear).pack(side="left", padx=6)
        ttk.Button(buttons, text="Submit feedback",
command=self.submit).pack(side="left")

        self.bind("<Alt-c>", lambda event: self.clear())
        self.bind("<Alt-s>", lambda event: self.submit())

    def clear_hint(self, _event=None):
        if self.comments.get("1.0", "end-1c") == "Optional: share as much or as
little as you wish.":
            self.comments.delete("1.0", "end")
            self.comments.configure(fg="black")

```

```

def clear(self):
    self.rating_box.set("Select an option")
    self.comments.delete("1.0", "end")
    self.comments.insert("1.0", "Optional: share as much or as little as you
wish.")

    self.comments.configure(fg="#555555")
    self.gender.set("")
    self.accessibility.set("")
    self.email.delete(0, "end")

def submit(self):
    if self.rating.get() == "Select an option":
        messagebox.showwarning("Required information", "Please select
an experience rating.")
        self.rating_box.focus_set()
        return
    messagebox.showinfo("Thank you", "Your feedback was submitted. Thank
you for helping us improve!")
    self.clear()

if __name__ == "__main__":
    FeedbackForm().mainloop()

```

Output:

 Inclusive User Feedback—□×

Share your feedback

Questions marked * are required. You may skip optional questions.

* How would you rate your experience?

What could we improve?

Gender identity (optional)

Accessibility needs (optional)

Email for a reply (optional)

We do not require your name, pronouns, or personal details.

Task-6: generate a Python program that predicts whether a person should be approved or rejected for a loan based on information such as income, age, employment status, credit score, and loan amount.

Analyze:

Check whether the AI-generated system makes decisions unfairly based on gender, religion, race, or other sensitive/unrelated personal attributes.

Input:

```
"""Task 6: Fairness in a loan approval system.
```

The first function demonstrates bias; the second uses only relevant financial

information and ignores protected attributes.

```
"""
```

```
def biased_decision(applicant):
```

```
    """Intentionally unfair example for comparison."""
```

```
    score = 0
```

```
    score += 2 if applicant["income"] >= 50000 else 0
```

```
    score += 2 if applicant["credit_score"] >= 680 else 0
```

```
    score += 1 if applicant["employment_status"] == "employed"
else 0
```

```
    score += 1 if applicant["age"] >= 25 else 0
```

```
    # These rules are discriminatory and are included only to
    expose the risk.
```

```
    if applicant["gender"] == "female":
```

```

        score -= 1
    if applicant["religion"] == "minority_religion":
        score -= 2
    if applicant["race"] == "minority_race":
        score -= 2
    return "Approved" if score >= 4 else "Rejected"

```

```

def fair_decision(applicant):
    """Approve based on financial factors, not gender, religion, or
    race."""
    score = 0
    score += 3 if applicant["income"] >= 50000 else 0
    score += 3 if applicant["credit_score"] >= 680 else 0
    score += 2 if applicant["employment_status"] == "employed"
    else 0
    score += 1 if applicant["debt_to_income"] <= 0.35 else 0
    score += 1 if applicant["loan_amount"] <=
    applicant["income"] * 3 else 0
    return "Approved" if score >= 7 else "Rejected"

```

```

def main():
    applicants = [

```

```
        {"name": "Alex", "income": 60000, "age": 35,
"employment_status": "employed",
        "credit_score": 720, "loan_amount": 120000,
"debt_to_income": 0.25,
        "gender": "male", "religion": "religion_a", "race":
"majority_race"},
        {"name": "Jordan", "income": 60000, "age": 35,
"employment_status": "employed",
        "credit_score": 720, "loan_amount": 120000,
"debt_to_income": 0.25,
        "gender": "female", "religion": "minority_religion",
"race": "minority_race"},
        {"name": "Casey", "income": 40000, "age": 29,
"employment_status": "employed",
        "credit_score": 650, "loan_amount": 90000,
"debt_to_income": 0.40,
        "gender": "nonbinary", "religion": "religion_a", "race":
"majority_race"},
    ]
```

```
print("Fairness risks identified:")

print("- Gender, religion, and race can cause direct
discrimination.")

print("- Biased historical data or proxy variables can create
disparate impact.")

print("- People with identical finances may receive different
decisions.")
```

```

    print("\nComparison (Alex and Jordan have identical
finances):")

    for applicant in applicants:

        print(f"{applicant['name']}:
biased={biased_decision(applicant)}, "
              f"fair={fair_decision(applicant)}")

    print("\nWhy the revised system is fairer:")

    print("It ignores protected attributes and uses income, credit
score, "
          "employment, debt burden, and loan size. Outcomes
should still be "
          "audited across groups for disparate impact.")

if __name__ == "__main__":
    main()

```

Output:

```

PS C:\Users\ADMIN\OneDrive\Documents\vs_code\ai.lab> C:\Users\ADMIN\AppData\Local\Python\pythoncore-3.14-64\python.exe
"c:/Users/ADMIN/OneDrive/Documents/vs_code/ai.lab/ailab6(task6).py"
Fairness risks identified:
- Gender, religion, and race can cause direct discrimination.
- Biased historical data or proxy variables can create disparate impact.
- People with identical finances may receive different decisions.

Comparison (Alex and Jordan have identical finances):
Alex: biased=Approved, fair=Approved
Jordan: biased=Rejected, fair=Approved
Casey: biased=Rejected, fair=Rejected

Why the revised system is fairer:
It ignores protected attributes and uses income, credit score, employment, debt burden, and loan size. Outcomes should st
ill be audited across groups for disparate impact.
PS C:\Users\ADMIN\OneDrive\Documents\vs_code\ai.lab> █

```

